Advanced Techniques for EEG Processing - Laboratory Exercises

Exercise 10 – Classification of motor imagery with a convolutional neural network from single EEG trials

In this exercise you are asked to train a Convolutional Neural Network (CNN) to classify 4 different imagined movements from single EEG trials. During the exercise, you will use a previously proposed CNN for motor imagery decoding, called *EEGNet* (https://pubmed.ncbi.nlm.nih.gov/29932424/), see Fig. 1, and you will train it on a common benchmark dataset (*BCI IV2a*: http://www.bbci.de/competition/iv/) for motor imagery decoding.

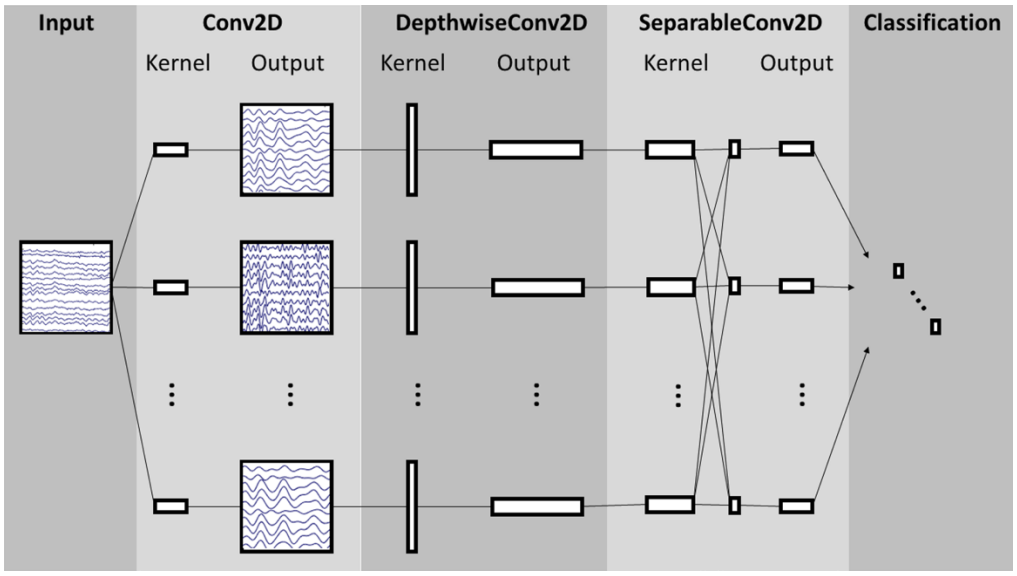

Fig. 1. EEGNet high-level scheme. The network input is a 3-D map of shape (channels, time, 1) containing a single EEG trial, while the network output is the predicted probability that the input trial belongs to one of the possible classes (C in total). The network processes the input by applying a sequence of trainable convolutions in time (1st layer: band-pass filtering of EEG), in space (2nd layer: spatial filtering), and again in time (3rd layer: deep recombination of the information in time). Then, neurons are densely connected with the output layer consisting of C neurons.

Signals were recorded from 22 sites at 250 Hz from 9 different subjects while they imagined to perform 4 different movements after a cue. The four imagery conditions were the imagination of movements of the left hand, right hand, feet, tongue. EEG signals were resampled at 128 Hz, band-pass filtered between 4 and 40 Hz, and epoched between 0.5-2.5 s respect to the movement onset of the movement imagination (2s-length epochs). For each subject, the dataset consists of a training and a test set, each formed by 288 EEG trials and the corresponding labels.

Each single EEG trial can be arranged as a 2-D map with channels along rows and time steps along columns. The CNN processes this map (input: EEG trial, channels=22 x time=256 x 1 input neurons, arranged in 3-D) producing the probability that the input EEG trial belongs to each imagined movement conditions (output: predicted probability distribution, 4 output neurons). The input has an additional third dimension as it consists of a single input feature map.

Create a new .py script and address the following points (1-5).

Notes. As for past Neural Systems (2022-2023) exercises, Python core packages as numpy (*import numpy as np*) and matplotlib.pyplot are needed (*import matplotlib.pyplot as plt*). The main keras (*from tensorflow import keras*) functionalities needed to solve the exercise are described in the lecture note "Keras for supervised learning: An introduction". Confusion matrices can be computed using scikit-learn (*from sklearn.metrics import confusion_matrix*). Lastly, in the *data_and_utils.zip* file are contained a mat file (*bci_iv2a_sub-008.mat*) containing the dataset (i.e., EEG trials and associated classes) of one subject, and a *utils.py* script, containing two functions needed for this exercise: *load_bci_iv2a* and *EEGNet (from utils import load_bci_iv2a, EEGNet)*. In case the training loop takes many seconds per epoch on your computer, you can use the free online service Google Colaboratory (enabling GPU in the "Edit→Notebook Settings" tab) to train your models faster (see https://colab.research.google.com/).

1) Data pre-processing and split.

1.a) Load the training and test sets for the provided subject with *load_bci_iv2a* method. The filepath to the mat file (bci_iv2a_sub-008.mat) must be provided as input parameter. The EEG pre-processing (briefly described above) was already performed. Labels are provided ordered to be associated to {'left hand', 'right hand', 'feet', 'tongue'} classes. Additional information are returned too (e.g., sampling rate, channel names, and imagined motor conditions), read the docstring for details

1.b) Add a singleton dimension (reshape from 22 x 256 to 22 x 256 x 1).

1.c) Check labels and transform them into one-hot labels if necessary (from $l \in \{0, \dots, 3\}$, e.g., $l = 2$ (label encoding) to the array representation $l = [0,0,1,0]$).

1.d) Extract a validation set from the overall training set (e.g., the first 10% of training examples).

1.e) Standardize inputs (for all sets) by using the mean value and standard deviation (across channels, time steps and epochs) computed on the training set.

1.f) Plot the histogram of labels, to analyze whether class unbalancing occurs (for all sets).

2) Network design. Define and initialize EEGNet using *EEGNet* method. To this aim the input shape and the number of classes (input and output definition of the network) must be provided to this function as input parameters, read the docstring for details. Inspect the network in its hidden layers and number of trainable parameters.

3) Network training.

3.a) Train the network by minimizing the cross-entropy loss function using stochastic gradient descent with momentum, using: learning rate=0.001, momentum term=0.9, batch size=64, max epochs=1000 (between 750-1500). Track also the accuracy in addition to the loss during training, on both training and validation sets. Save only the best model on the validation loss, overall, during training.

3.b) Analyze the convergence and the proper fit by plotting the training and validation losses (/accuracies) over epochs.

4) Network evaluation. Load the saved best model. Evaluate the model (i.e., compute the predicted output probabilities) on the training, validation, and test sets. Compute the predicted labels as the labels with the highest output probability. Compute the confusion matrices and accuracies on all sets.

5) Network visualization of spatial features. The convolution in the second convolutional layer of EEGNet is performed only in the spatial domain, and, thus, the learned convolutional filters can be viewed as arrays with shape channels x 1 (16 filters in total). Due to this characteristic, we refer to these filters as "spatial filters", and each one of these filters is used by the CNN to combine the input feature maps (band-pass filtered versions of the input EEG trial) across electrodes to properly classify the 4 imagined movements. By extracting these spatial filters, we can understand which are the electrodes that were weighted more by the CNN to discriminate between the 4 motor classes.

5.a) Access the learned spatial weights from the loaded network in point 4. Due to a "special" convolution used in EEGNet to operate in space, the weights have shape (22, 1, 8, 2), where the spatial filters (16) are spread across the last two dimensions.

5.b) Compute the absolute value of spatial weights.

5.c) Average all spatial weights together, obtaining an absolute score of "importance" associated to each EEG channel. This score resumes the discriminatory power in the addressed task (i.e., left hand vs. right hand vs. tongue vs. feet imagined movements), overall, for each EEG channel.

5.d) Plot the average absolute spatial weights (barplot) and rank the EEG channels based on the resulting "importance" score. It is worth remarking that this ranking could also be mapped onto the scalp by using 2-D topoplots (for semplicity here it is suggested a simple barplot). If the network is properly trained, it should emerge that, among the most used electrodes, central / centro-parietal electrodes (covering approx. motor areas) are the most used to classify the imagined motor conditions.